# ReactJS

# Day -1

# Introduction

- **ReactJS** is **JavaScript library** created by **JORDAN WALKE** at **Facebook** in 2011, and was released to public in May 2013.

*Jordan Walke,*
*@FaceBook*

- **ReactJS** is a **declarative, efficient,** and **flexible JavaScript library** for building reusable UI components.
- **ReactJS** though developed and maintained by Facebook, was later used in products like **WattsApp** & **Instagram.**
- **ReactJS** uses **declarative paradigm** making it easier to reason about application.
- **ReactJS** is not a **framework**.

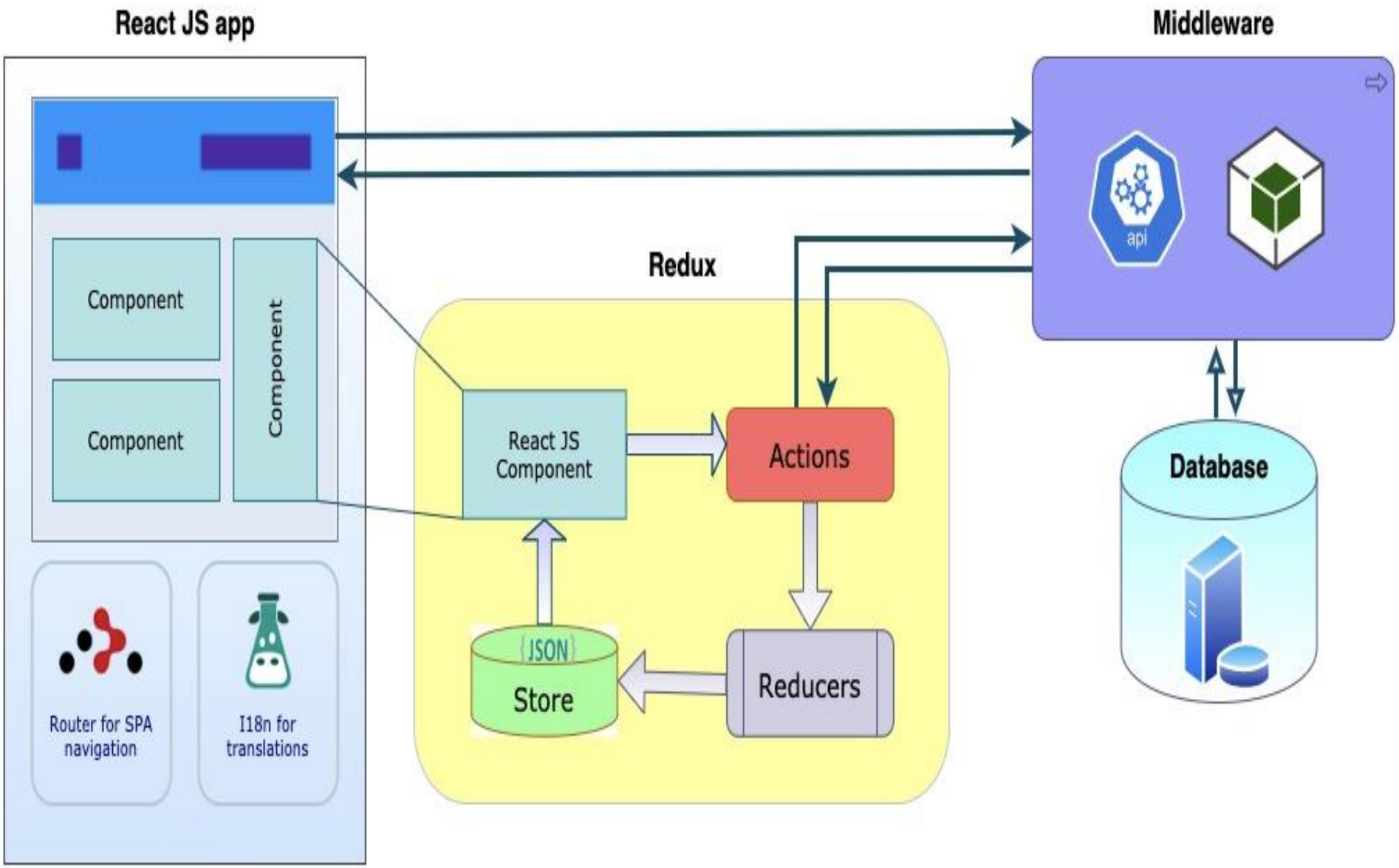
# React vs Angular vs Vue

- React is used for building dynamic web applications.
- React is a **library focused** on the **view layer,** providing more flexibility in terms of integrating with other tools

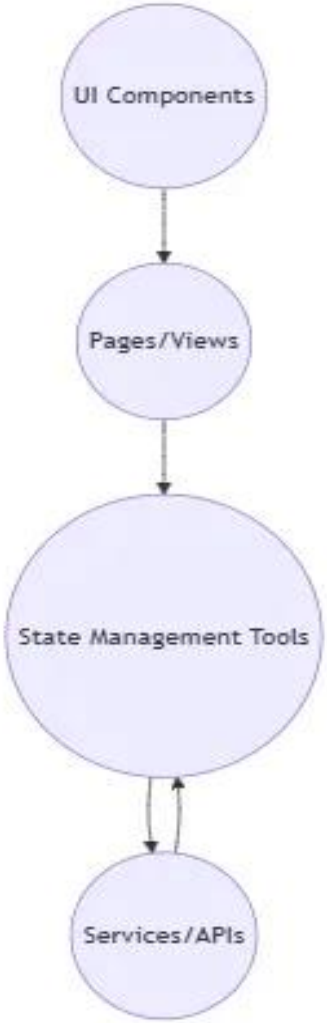
- Angular is used for building dynamic web applications.
- Angular, on the other hand, is a **comprehensive framework** that comes with an **opinionated structure** and a set of tools for building large-scale applications.

- Vue is a **progressive framework**.
- Vue offers a **more opinionated** and **structured approach** with built-in solutions for many common development needs (e.g., state management with Vuex/Pinia, routing with Vue Router).

# Top-level architecture



# contd..


UI Components
Pages/Views
State Management Tools
Services/APIs

# Core Components of React

- **UI Components:** They are building blocks (buttons, forms, cards, etc.). They encapsulate the structure and behavior of the elements listed below:
  - ✓ **Buttons**: Trigger actions like form submissions or navigation.
  - ✓ **Forms**: Collect and validate user input.
  - ✓ **Cards**: Display grouped information in a clean, structured manner.
  - ✓ **Reusable Design**: React promotes breaking down UI into smaller components, like a Submit button used across multiple pages.
- **Pages/Views:** They combine multiple components and define higher-level screens.
  - ✓ **Routing**: React Router helps define different views based on URL paths (e.g., home, profile, dashboard).
  - ✓ **Dynamic Content**: Views dynamically render content based on state or <u>props</u>, providing a seamless user experience.
  - ✓ **Hierarchy**: Combine multiple UI components to create comprehensive and consistent views.

# contd..

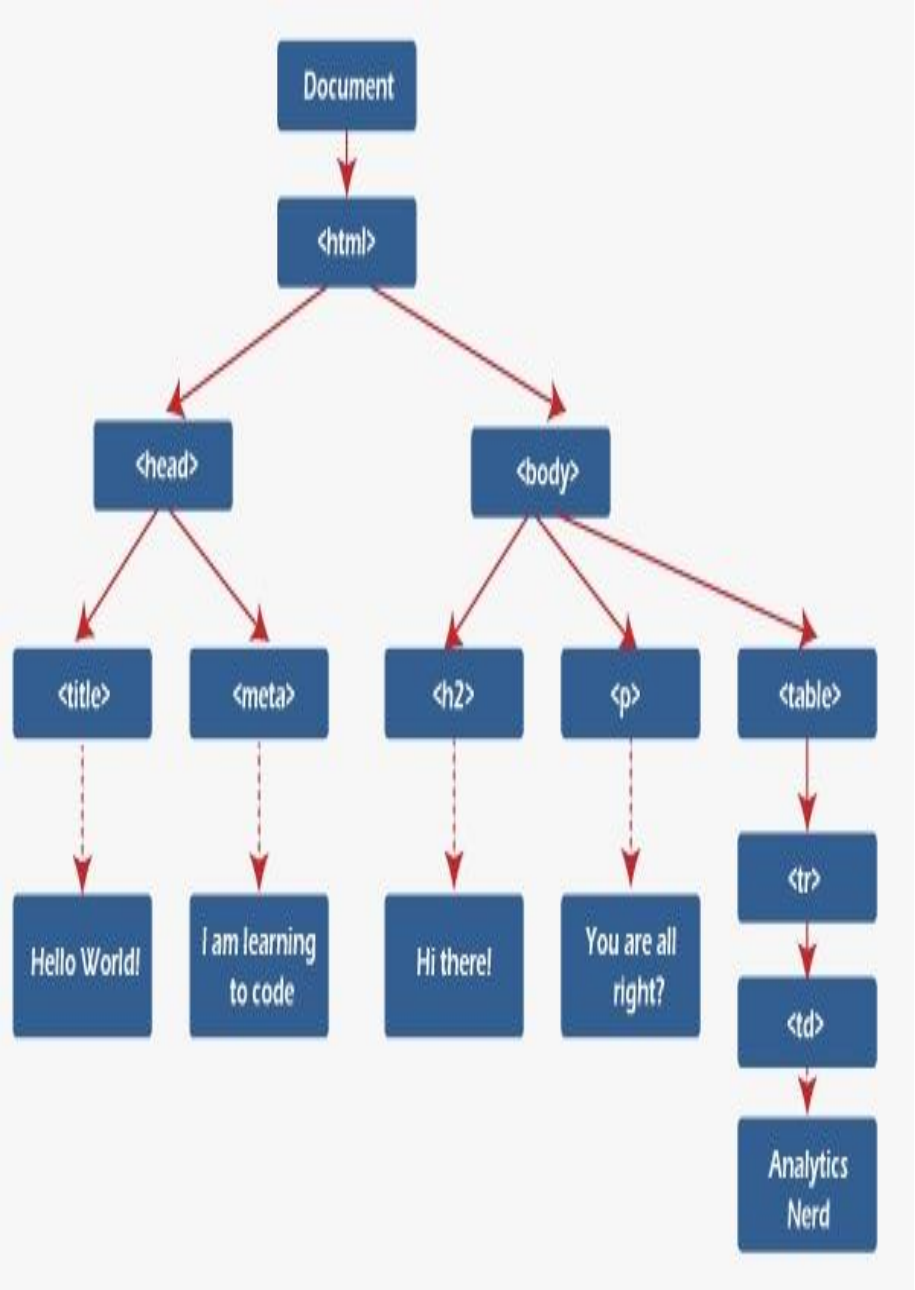
- **State Management –** state management tools centralize and streamline the handling of application-wide state, ensuring consistent data flow and interaction across components.
  - ✓ **Context API**: Native React tool for passing data through the component tree without props drilling.
  - ✓ **Redux**: A robust library for managing complex application states with predictable behavior.
  - ✓ **Zustand**: A simpler, lightweight alternative to Redux for state management.

  These tools help synchronize the UI with dynamic data changes, improving application reliability.
- **Services/APIs:** They handle external data fetching or business logic (e.g., database calls and authentication).
  - ✓ **API Integration**: Fetch or send data using libraries like Axios or Fetch API.
  - ✓ **Authentication**: Manage user login sessions, permissions, and security checks.
  - ✓ **Custom Logic**: Encapsulate reusable logic, like calculations or data transformations, in service layers to reduce redundancy.

# contd..

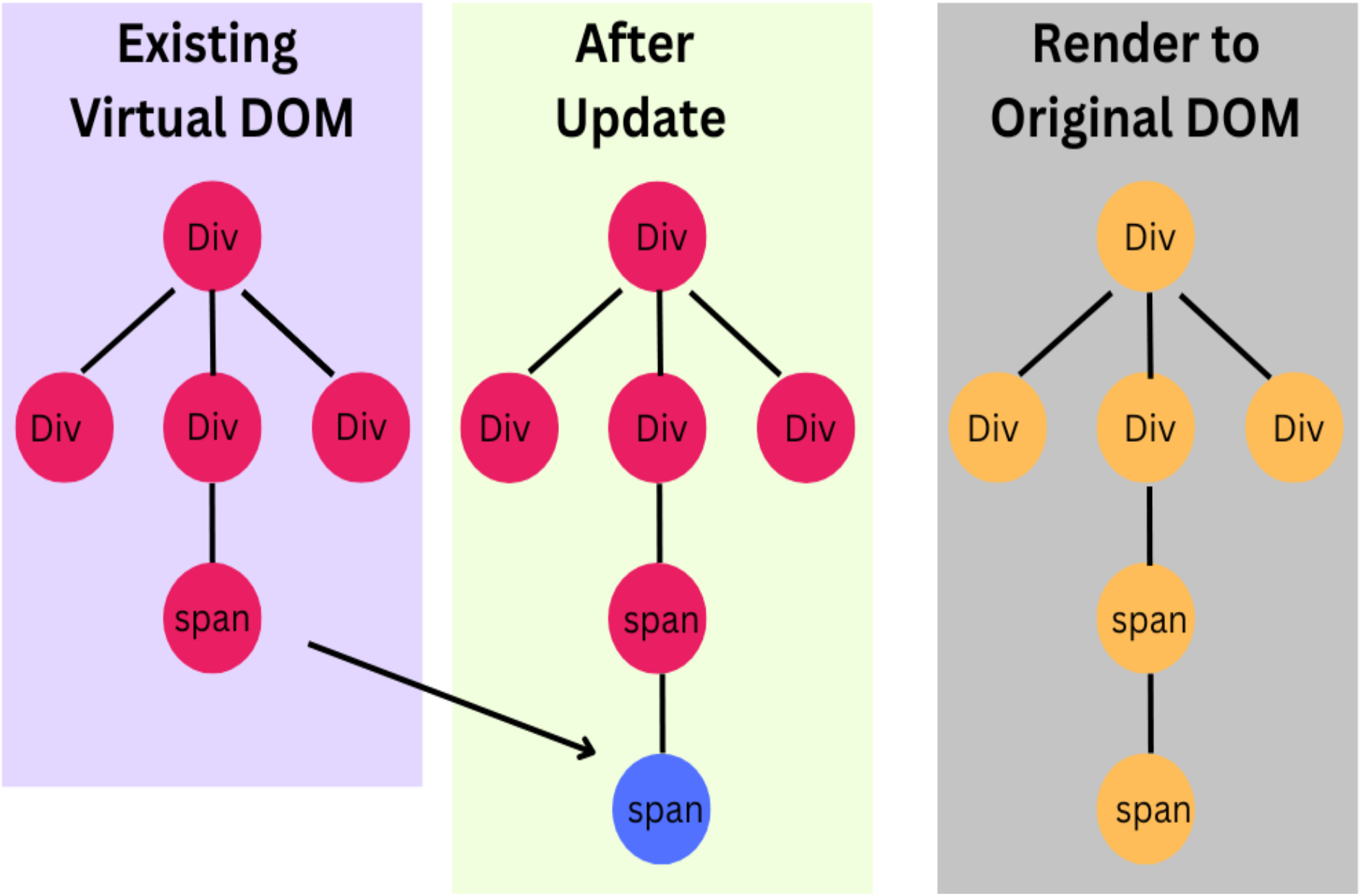
- **Hooks in React JS** simplify functionality and reduce repetitive code by enabling state and lifecycle management in functional components.
  - ✓ **useState**: Manage local component state.
  - ✓ **useEffect**: Handle side effects like API calls or DOM updates.
  - ✓ **useContext**: Access global state from Context API.
  - ✓ **Custom Hooks**: Encapsulate reusable logic, such as fetching data and improving maintainability and scalability.
- **Babel** is a JavaScript compiler that takes modern JavaScript code and returns browser compatible JavaScript. It translates JSX into React.createElement calls.
- React application is made of **multiple components**, each responsible for rendering a **small, reusable** piece of html.
- **React Component** – is a JavaScript/TypeScript class (extends the React Component class) or a pure JavaScript/TypeScript function. **Components** have **properties, state management, lifecycle & event-handlers** to handle any logic. Components use React Elements and JSX to design their individual user interface.

# DOM – Document Object Model

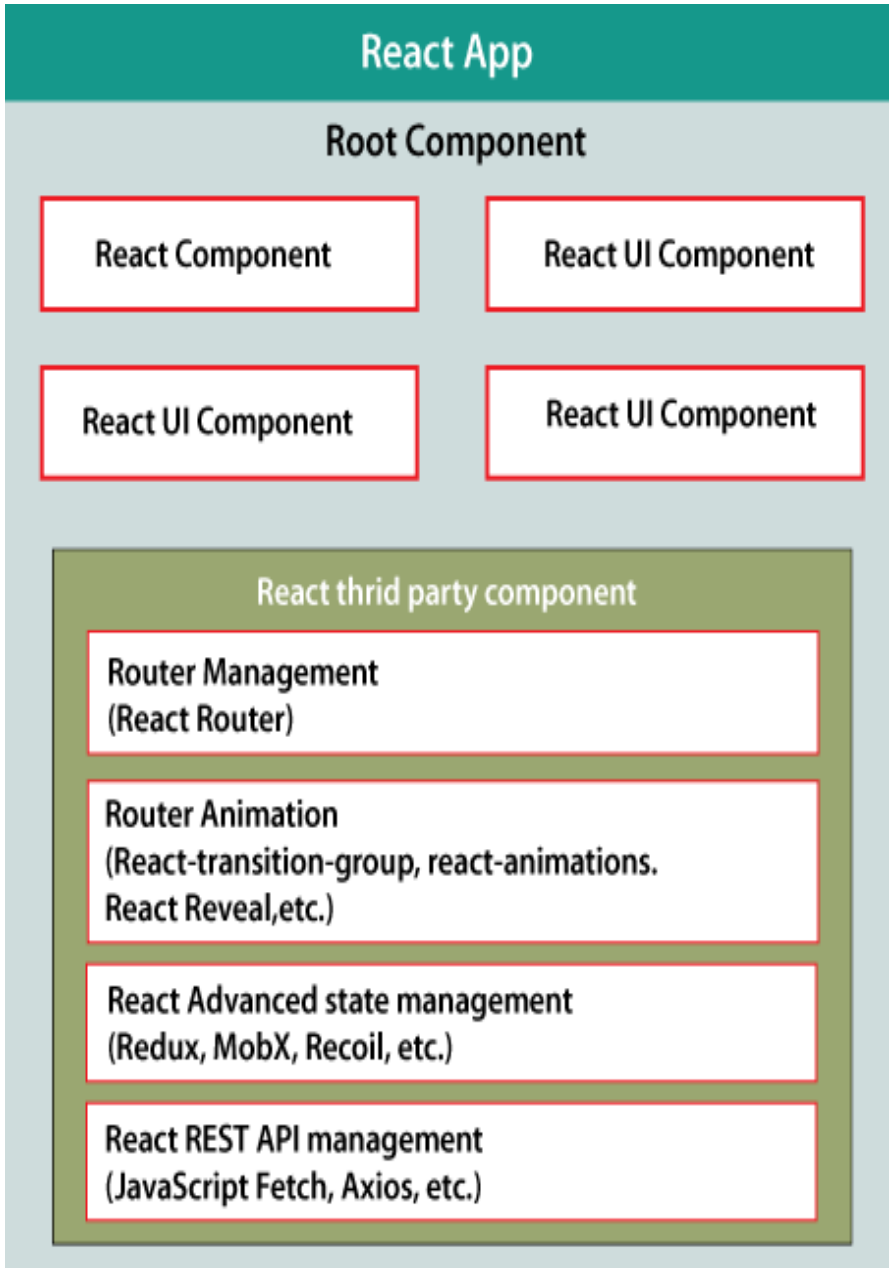



- DOM - <u>Document Object Model</u>, represents the entire UI(**User Interface**) of the *web application as the tree data structure.*
- React uses **Virtual DOM** which is an exact copy of **Real DOM**.
- JavaScript updates whole DOM once, making the web application slow.
- Whenever there is a **modification** in the **web application**, the whole **virtual DOM** is **updated first** and finds the difference between real DOM and Virtual DOM.
- The DOM is constituted as a tree data structure. It consists of the node for each **UI element** present in the web document.

# Virtual DOM


Existing Virtual DOM
Div
Div
Div
Div
span
After Update
Div
Div
Div
Div
span
span
Render to Original DOM
Div
Div
Div
Div
span
span

# Components




- React app starts with **single root component**.
- The core components is/are created by using one or more components.
- Most of the components are user interface components. Each component can be nested with any other component at any level.
- Component is built by composing smaller components rather than by inheritance. This is called **Composition**.
- React apps may include third-party components for a specific purpose, such as **routing, animation, state management**, etc.

# Getting started

- **IDE –** Visual Studio Code.
- ReactJS designs **views** for each **state** in **application**, and **update** & **render** just the **right component** when the data changes. The **declarative view** makes code **more predictable** and easier to debug.
  - ✓ **npm view react version**
  - ✓ **npx create-react-app** my-app1
  - ✓ **npx create-react-app** my-app1 --template typescript
- **import React from 'react'**

imports the entire React library, making its functionalities and features available within the current file. Includes core features – creating components, managing state, and handling lifecycle methods.

when React is imported, the user-code is told to make use of functions and objects provided by React library.

- **import * as React from 'react'**

imports all **named exports** from the React library as an object

# create-react-app

- **NodeJS** and **npm** are the platforms needed to develop ReactJS application.
- **create–react–app** is a tool maintained by Facebook to install react. Using this, there is no need to manually having to deal with transpiling tools like webpack and babel.
- Install React using npm package manager.
- The **create–react–app** package will take care of complexity of ReactJS installation.

  **c:\Temp–React\>npm install -g create–react–app**

- After installation, create a new react project using **create–react–app** command.

  **c:\Temp–React\>npx create–react–app** Sample-reactapp1

- **npx** is **package runner tool** from **npm** versions ***5.2*** and above.

  **c:\Temp–React\ Sample-ReactApp1 >npm start**

# Creating react project with vite

File Edit Selection View ···

Search

EXPLORER

OPEN EDITORS

NO FOLDER OPENED

OUTLINE

TIMELINE

LINECOUNTER

MAVEN

OUTPUT DEBUG CONSOLE TERMINAL PORTS PLAYWRIGHT

TERMINAL node

```
PS C:\react-scan-it> npm create vite@latest

  > npx
  > create-vite

  ◇  Project name:
  │  test-react-1
  │
  ◆  Select a framework:
  │  ○ Vanilla
  │  ○ Vue
  │  ● React
  │  ○ Preact
  │  ○ Lit
  │  ○ Svelte
  │  ○ Solid
  │  ○ Ember
  │  ○ Qwik
  │  ○ Angular
  │  ○ Marko
  │  ○ Others
```

0 0 Go Live

Type here to search

ENG 12:11 PM 2/22/2026

# Folder Structure

File Edit Selection View ··· react-scan-it

EXPLORER

OPEN EDITORS
- App.jsx test-react-1\src

REACT-SCAN-IT
- test-react-1
  - node_modules
  - public
  - src
    - assets
    - App.css
    - App.jsx
    - index.css
    - main.jsx
  - .gitignore
  - eslint.config.js
  - index.html
  - package-lock.json
  - package.json
  - README.md
  - vite.config.js

OUTLINE
TIMELINE
LINECOUNTER

App.jsx

test-react-1 > src > App.jsx > ...

```jsx
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from '/vite.svg'
import './App.css'

function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <div>
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS PLAYWRIGHT

TERMINAL powershell - test-react-1

```
PS C:\react-scan-it\test-react-1>
```

0 0 Ln 1, Col 1 Spaces: 2 UTF-8 LF {} JavaScript JSX Go Live

Type here to search ENG 2:41 PM 2/22/2026

# contd..

1. **node_modules:** Contains the React library and other third party libraries.
2. **public:** Holds the public assets of the application. It contains the index.html where React will **mount the application** by default on the ** ** element.
3. **src:** Contains the **App.css, App.js, App.test.js, index.css, index.js,** and **serviceWorker.js** files. App.js file displays output screen.
4. **package-lock.json:** Its generated automatically for any operations where npm package modifies either the node_modules tree or package.json. It cannot be published.
5. **package.json:** It holds various metadata required for the project. It gives information to npm, which allows to identify the project as well as handle the project?s dependencies.
6. **README.md:** It provides the documentation to read about React topics.

# contd..

File Edit Selection View Go Run ··· reactproject

EXPLORER

OPEN EDITORS

REACTPROJECT

reactproject_1

OUTPUT DEBUG CONSOLE TERMINAL PORTS

TERMINAL node

```
> npx
> create-vite my-app react

◆ Select a framework:
  ○ Vanilla
  ○ Vue
  ● React
  ○ Preact
  ○ Lit
  ○ Svelte
  ○ Solid
  ○ Qwik
  ○ Angular
  ○ Marko
  ○ Others
```

OUTLINE

TIMELINE

LINECOUNTER

master ⊗ 0 ⚠ 0 Go Live

Type here to search ENG 1:51 PM 11/26/2025

# Folder Structure

File Edit Selection View Go Run ··· reactproject

EXPLORER

OPEN EDITORS

REACTPROJECT

- my-app
  - node_modules
  - public
    - vite.svg
  - src
    - assets
    - App.css
    - App.tsx
    - index.css
    - main.tsx
  - .gitignore
  - eslint.config.js
  - index.html
  - package-lock.json
  - package.json
  - README.md
  - tsconfig.app.json
  - tsconfig.json
  - tsconfig.node.json
  - vite.config.ts
- reactproject_1

OUTLINE

TIMELINE

LINECOUNTER

OUTPUT DEBUG CONSOLE TERMINAL PORTS

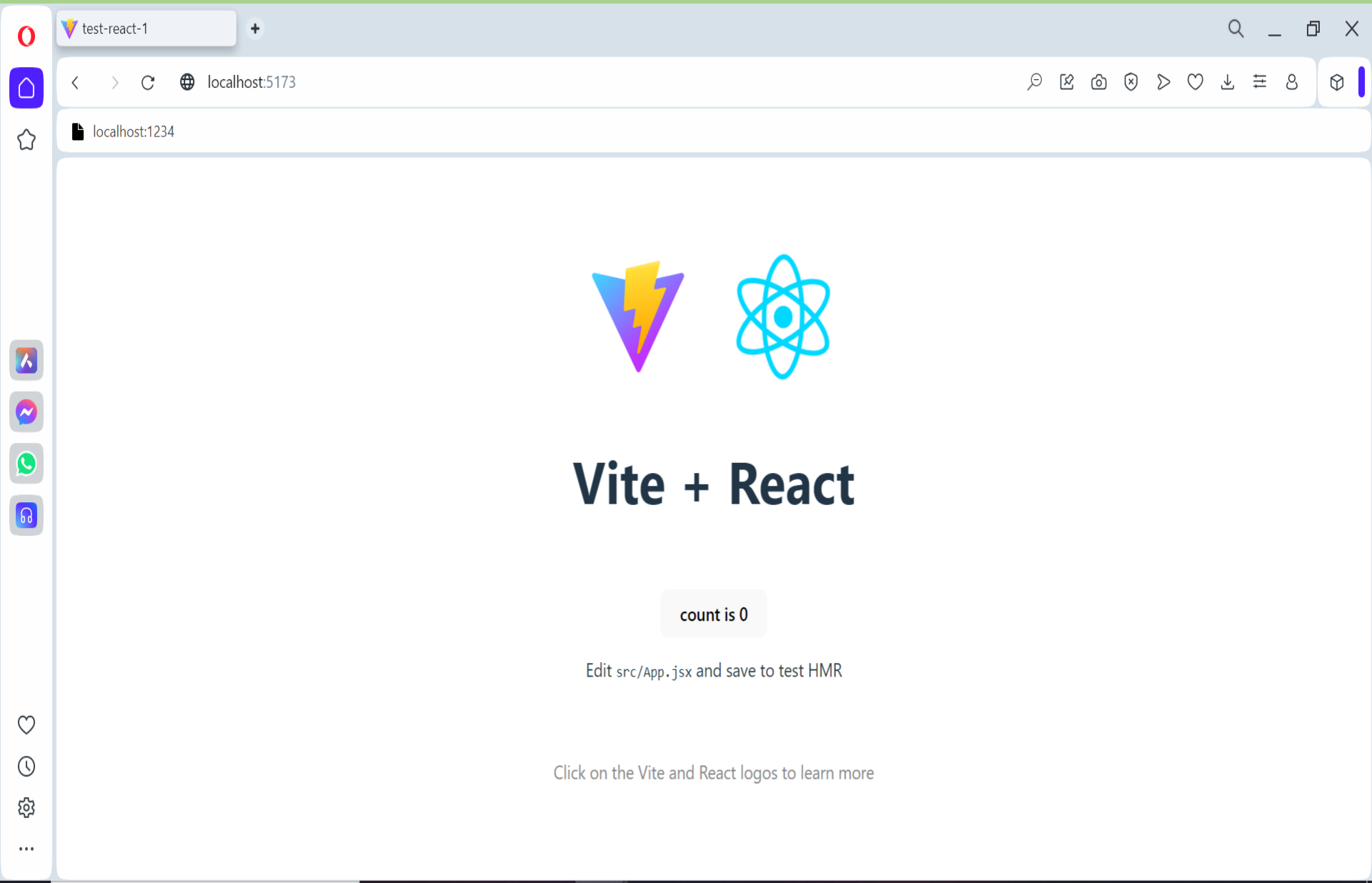
TERMINAL

```
◇ Installing dependencies with npm...

added 181 packages, and audited 182 packages in 20s

48 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

◇ Starting dev server...

> my-app@0.0.0 dev
> vite
```

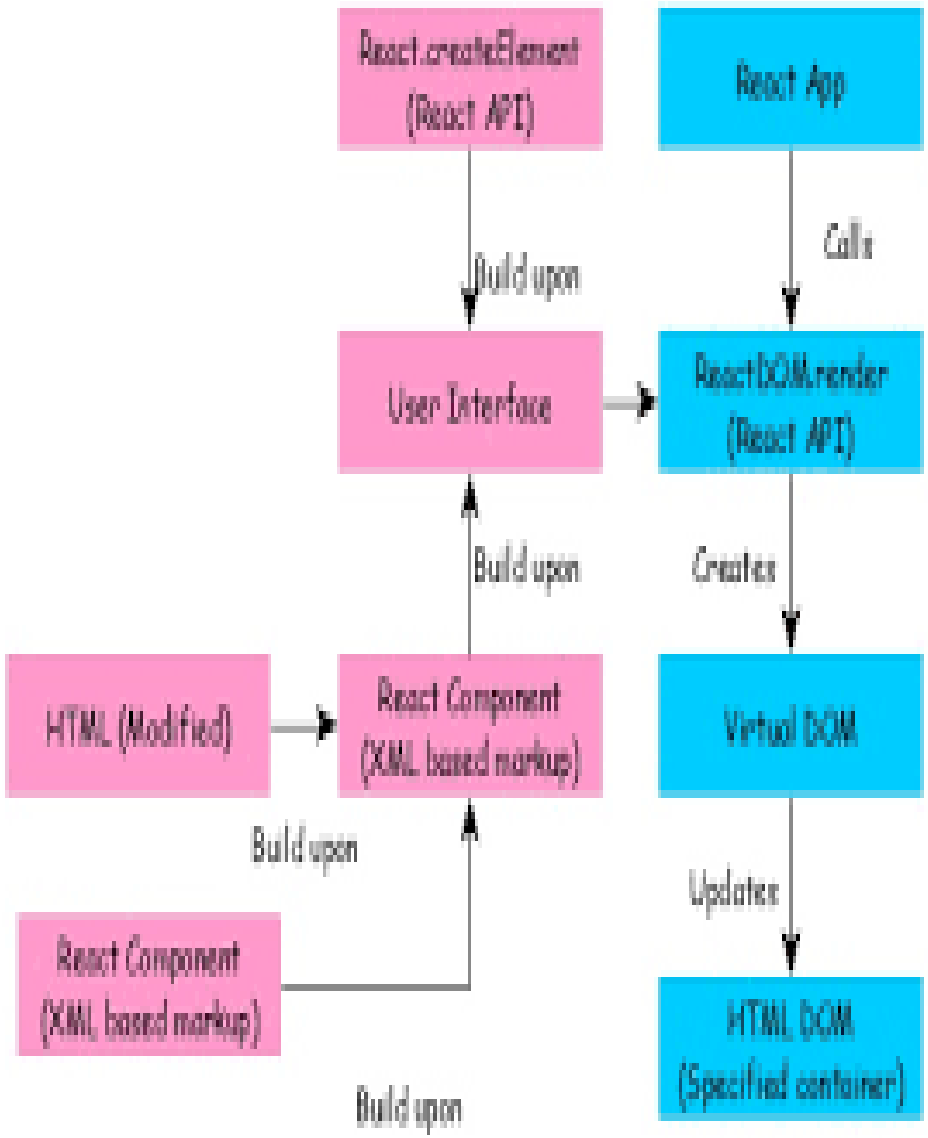
master ⊗ 0 ⚠ 0 Go Live

Type here to search ENG 2:03 PM 11/26/2025

# Output on Browser



# Flow Diagram

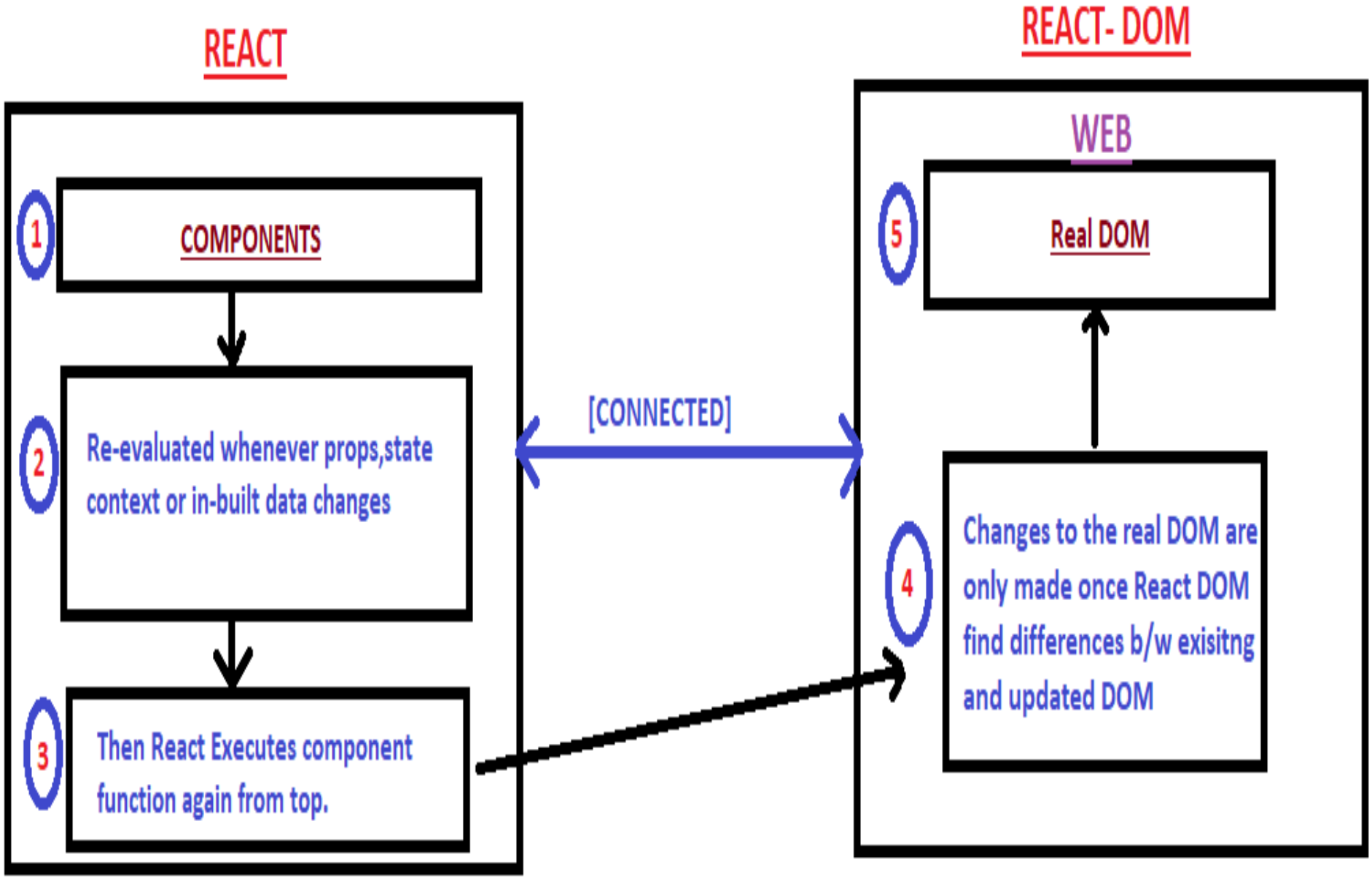



- The React app calls the **ReactDOM.render()** method, bypassing the user interface created using the React component (coded in **JSX** or **React Element** format) and the container to render the user interface.
- **ReactDOM.render()** processes JSX or React Elements and emits virtual DOM.
- The virtual DOM will be merged and rendered in the container.

# ReactDOM

- Virtual DOM is a DOM representation of UI in JavaScript and it will be stored in a memory to make comparisons when the render is called.
- The **Virtual DOM** can be referred to as a copy of the actual DOM representation.
- Re–rendered DOM is one which gets created when the render method is being called and it will be used to compare with virtual DOM.
- It holds the updates made by the user and finally reflect it over to the original Browser DOM.
- **ReactDOM** is a package that provides DOM specific methods.
- The package is used at the top level of a web app to enable an efficient way of managing DOM elements of the web page.

# contd..

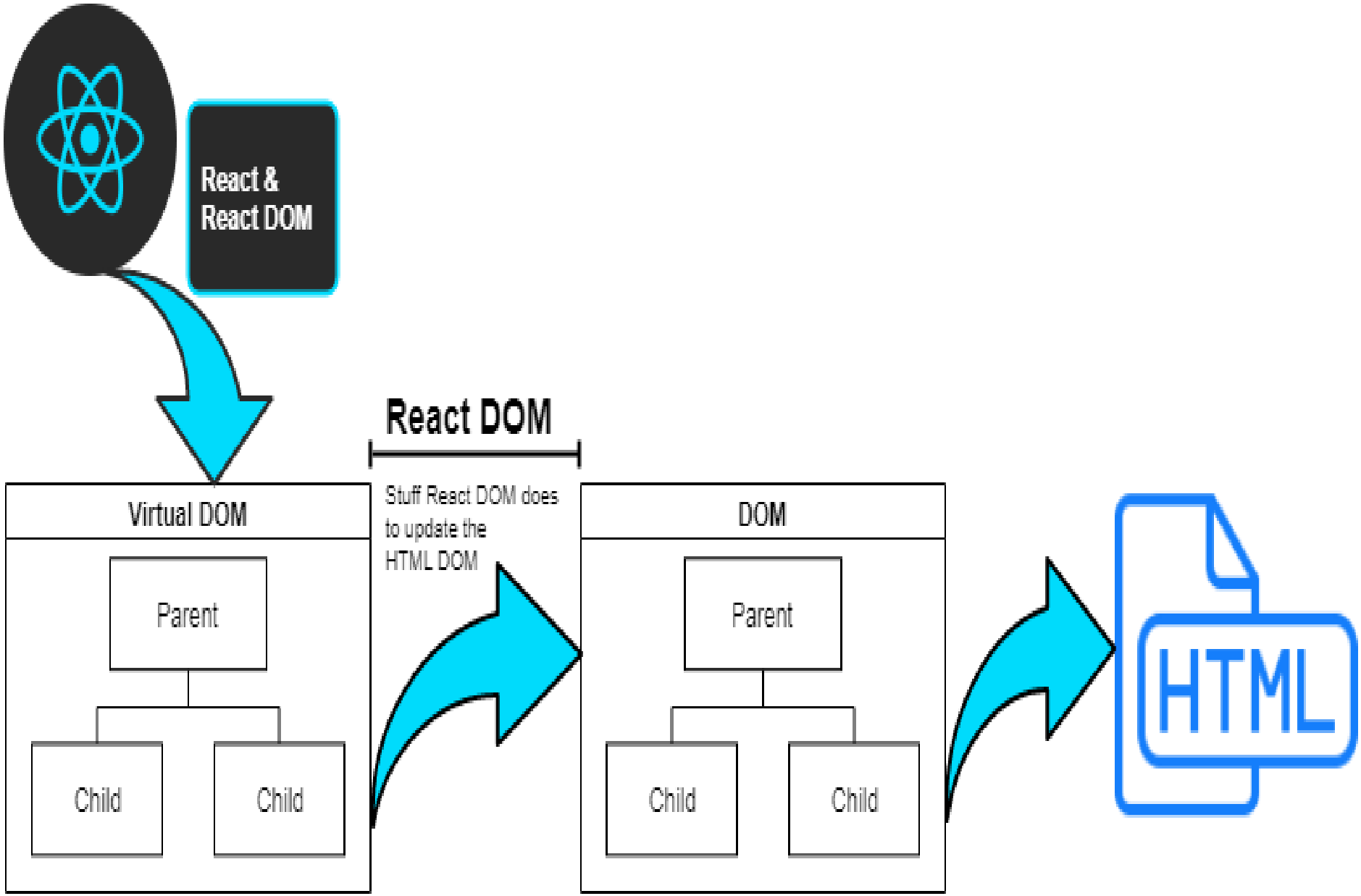

REACT
REACT- DOM
WEB
1
COMPONENTS
2
Re-evaluated whenever props,state context or in-built data changes
3
Then React Executes component function again from top.
[CONNECTED]
5
Real DOM
4
Changes to the real DOM are only made once React DOM find differences b/w exisitng and updated DOM

# How DOM updates in React

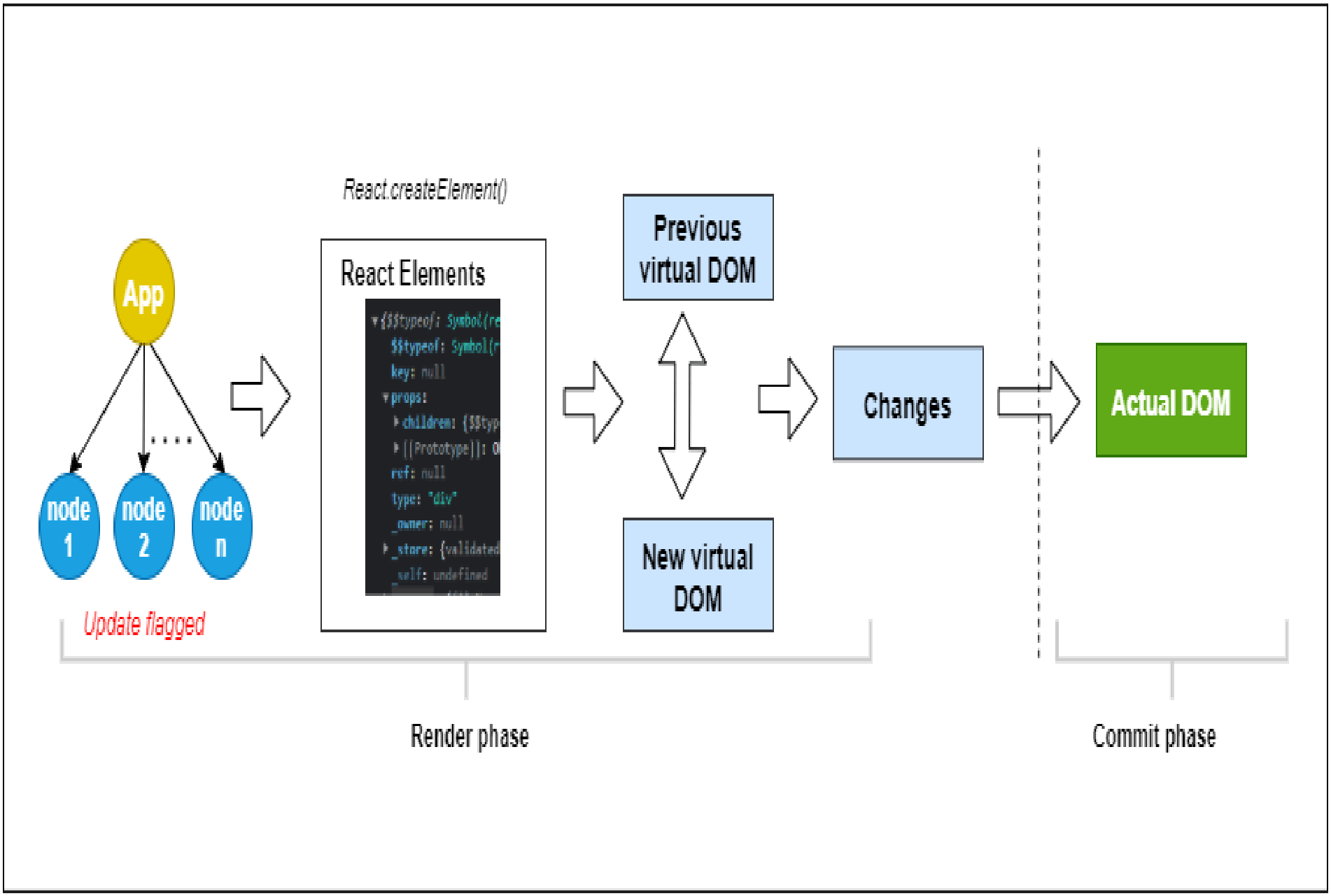
- **ReactDOM** provides the developers with an **API** containing the following methods and a few more.
    a. render()
    b. findDOMNode()
    c. unmountComponentAtNode()
    d. hydrate()
    e. createPortal()
    f. Return Type: This function returns a reference to the component or null if a stateless component was rendered.
- On the first run, both virtual DOM and real DOM tree are created

1. The nodes in virtual DOM are updated. (React works on observable patterns, hence, whenever there is a change in the state).
2. Then, react compares virtual DOM with real DOM and updates the changes. This process is called **reconciliation**.
3. React uses a heuristic algorithm called the Diffing algorithm for reconciliation based on these assumptions:

# contd..


React &
React DOM
React DOM
Virtual DOM
Parent
Child
Child
Stuff React DOM does
to update the
HTML DOM
DOM
Parent
Child
Child
HTML

# contd..


App
node 1
node 2
node n
Update flagged
React.createElement()
React Elements
Previous virtual DOM
New virtual DOM
Changes
Actual DOM
Render phase
Commit phase

# React elements

- The **React.createElement()** method is a tool for creating React elements.
- React elements are the building blocks of React applications, representing the nodes of the virtual DOM that React uses to efficiently update and render user interfaces.
- The createElement() method provides deeper insight into how React works under the hood and allows for more dynamic and programmatic element creation.
- A <u>React Element</u> is the smallest building block of a React app and it describes what you want to see on the screen. Not to be confused with browser DOM elements, React Elements are plain objects and are cheap to create.
-  <h1>Hello, World!</h1>

- React.createElement( "div", { className: "App" },
  React.createElement("h1", null, "Hello, World!")

# contd..

- **createElement** lets create a React element. It serves as an alternative to writing JSX. createElement method is a fundamental API used to create React elements, which are lightweight representations of DOM elements or custom components.
- **Syntax and Usage**
- **createElement(type , props , …children)**
- The createElement method takes three primary arguments:
- **Type**: The type of the element to create. This can be a string representing an HTML tag name (e.g., 'div', 'span'), a React component (class or functional component), or a React fragment (React.Fragment).
- **Props**: An object containing properties (attributes) to assign to the element or component. These can include event handlers, styles, or custom data.
- **Children**: Optional additional arguments that represent the children of the element. These can be further React elements, strings, or numbers.

# JSX – JavaScript XML

- JSX *is a syntax extension for JavaScript that lets you write HTML-like markup inside a JavaScript file.*
- JSX (JavaScript XML) syntax is more commonly used for its readability and conciseness.
  - JSX is a syntax extension for JavaScript that lets developer write HTML – like markup inside a JavaScript file.
  - JSX is Facebook's xml–like syntax extension to JavaScript, not a language per se unlike typescript which is a language a syntactic–superset of JavaScript.
  - JSX is a syntax extension of regular JavaScript and is used to create React elements. These elements are then rendered to the React DOM.
  - React code is written in JSX. Its faster than normal JS as it performs optimizations while translating to regular JS making it easier to create templates.
  - Normal JS expressions with JSX. To embed any JavaScript expression in JSX code, wrap that expression in **curly braces {}**.
  - JSX allows to use attributes with the HTML elements. However unlike the naming convention of HTML, JSX uses camel case convention for attributes

# React Hooks

# React Hooks

- **React Hooks** are **JavaScript functions** introduced in React 16.8 that allow developers to "hook into" **React state** and **lifecycle features** directly from functional components.

Note – Prior to Hooks, these features were primarily available only in class components.

- Key aspects of React Hooks –
  - ✓ Enable state and lifecycle management in functional components.
  - ✓ Promote code reusability and organization.
  - ✓ Simplify component structure.
  - ✓ Support custom Hooks.
- Commonly used built-in React Hooks include:
  - ✓ useState
  - ✓ useEffect
  - ✓ useContext
  - ✓ useRef
  - ✓ useCallback
  - ✓ useMemo:

# useState()

- **useState** hook allows **functional components** to **manage state**. It returns a **stateful value** and a **function to update** it and works by handling and managing state in applications.
- **useState()** hook can conveniently hold strings, arrays, numbers, objects and much more.
- **useState** with TypeScript, provides **type inference** based on the initial value provided or allows developers to specify the **type explicitly using generics.**

  **Step 1 :** import { useState } from 'react';

  **Step 2 : useState()** hook takes the first (initial) value of the state variable as its argument. The second value then sets user state, initiated as setState.
- E.g.
  - ✓ const [state, setState] = useState(initial values goes here)
  - ✓ const [calories, setCalories] = useState(initial value of calories)

# contd..

```
1. onhandleChange = (event) => {
2. const { target: { name, value } } = event
3. this.setState({ [name]: value, event: event })
4. }
```

**Line 2 :** changing an input field created an event object and that **line 2** was destructuring that event object into two separate variables, **name** and **value**,

**Line 3 :** this.setState take a key as an array

**Note ->** In Javascript, when you create an object literal {} and you wrap the object's key in array brackets [key] you can dynamically set that key.

# contd..

- **const [ename, setEname] = useState('Vikas')** - ename is inferred to be of type string because an initial value of 'John' is provided to useState.
- **const [ename, setEname] = useState<string>()** – ename is of type string | undefined. This is useful when the initial value is not provided, and user wants to enforce a specific type
- **const [enames, setEnames] = useState([])** – enames is inferred to be of type never[] because an empty array is provided, and TypeScript cannot infer the type of its elements.
- Inferring **types** and provide **explicit type annotations** is essential for writing type-safe React components.
- Leveraging the power of TypeScript, developers can catch **type-related errors** early in the development process, leading to more **robust** and **maintainable codebases.**

# contd..

- Declaring **const [state, setState] = useState()** and still be able to change the **const state** even though it's **const? const state** works fine even though it's **not in different scope** when reassigned with **setState**.
- When **setState()** is called, the **const state does *not* change**. Instead the **component is called again**, with **useState** now returning **updated value**. Value assigned to **state const** is of **next-function invocation**. Each time the component is rendered, new value is got and assigned to **new** const variable.
- **Components** are **functions** that **get called** on **re-renders**, while React manages to handle unchanged and changed states so that resources are calculated economically. class components **can't use hooks**, since classes aren't **'called'** on every re-render.
- **const** means value of variable **cannot** be **re-assigned** using assignment operator. This is true with react state since state is immutable in react. Hence **setState function** to change it.

# contd..

- The line const[formData, setFormData] = useState({}) uses **array destructuring** in JavaScript. This is possible because the useState hook, when called, returns an array containing two elements:
- The **current state value**, which is formData in your example, initialized to an empty object {}.
- A **state updater function**, which is setFormData, used to update the state and trigger a re-render of the component.
- useState({}) initializes a <u>React state variable</u> as an empty object ({}). It is a hook used in functional components to manage data that changes, allowing the component to re-render when the object is updated. The {} sets the initial value, enabling you to add properties later.

# contd..

- **Key Details**
- **Initial State:** The {} inside useState() means the state starts as an empty object.
- **Purpose:** It is commonly used when you need to store multiple related values, such as form input fields, rather than just a single string or number.
- **Usage:** Usually, it is destructuring like this: const [data, setData] = useState({});.
- **Updating:** To update an object state, you must pass a new object, often using the spread operator to retain previous values: setData({ ...data, key: value });.

# useEffect()

- **useEffect** allows to perform **side effects** in function components. E.g. **fetching data** or **setting up event listeners, updating DOM,** or **using timers.**
- **useEffect(()=>{}, [])**
- **useEffect** takes two arguments – a function that contains the side effect logic, and an array of dependencies that controls when the effect runs.
- **useEffect** also fetch data from an API when the component is rendered. User passes empty array as the second argument which tells React that the effect doesn't depend on any values from props or state, so it only needs to be run once when the component is mounted.
- When the data is returned from the API, update the component's state with setData, which causes the component to re-render.
- A list of values can be passed as the second argument to useEffect, which tells React to re-run the effect whenever one of those values changes.

# useMemo()

- **useMemo** is a powerful tool that allows to optimize performance by **memoizing** the **result** of **expensive computations**.
- **useMemo** when used in conjunction with TypeScript, provides type safety and helps prevent common runtime errors.
- **useMemo** takes **two arguments**: a **function** that computes a value and an **array of dependencies**. The hook returns a memoized version of the computed value, which remains consistent across renders provided dependencies do not change.

  E.g.
  ```
  const sum = useMemo(() => {
    return data.reduce((acc, val) => acc + val, 0); }, [data]);
  ```
- **useMemo** can memoize the rendering of complex or expensive components. Useful when condition-based rendering of components.
- **useMemo** can be used to prevent unnecessary re-renders of child components. When passing computed or derived values as props to child components, memoizing those values can avoid unnecessary updates in the child components.

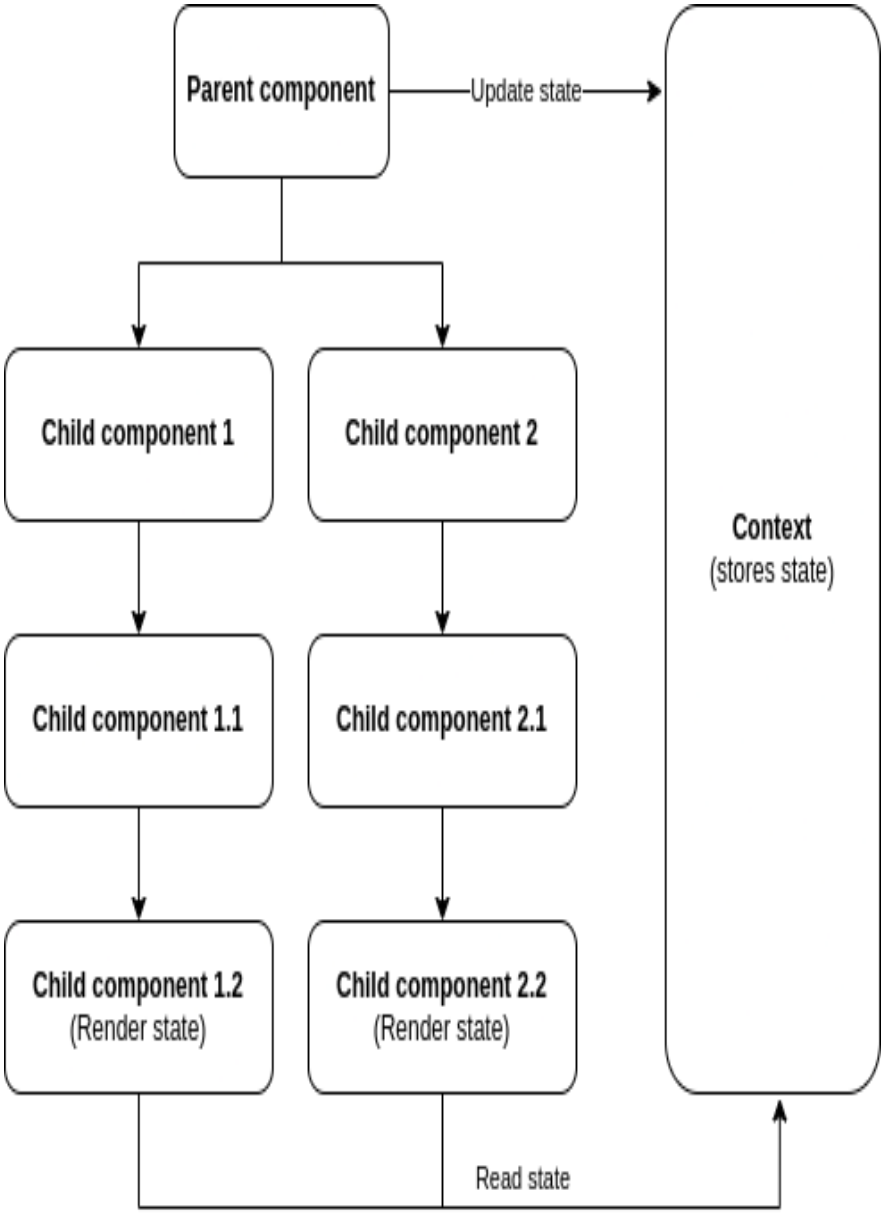
# contd..

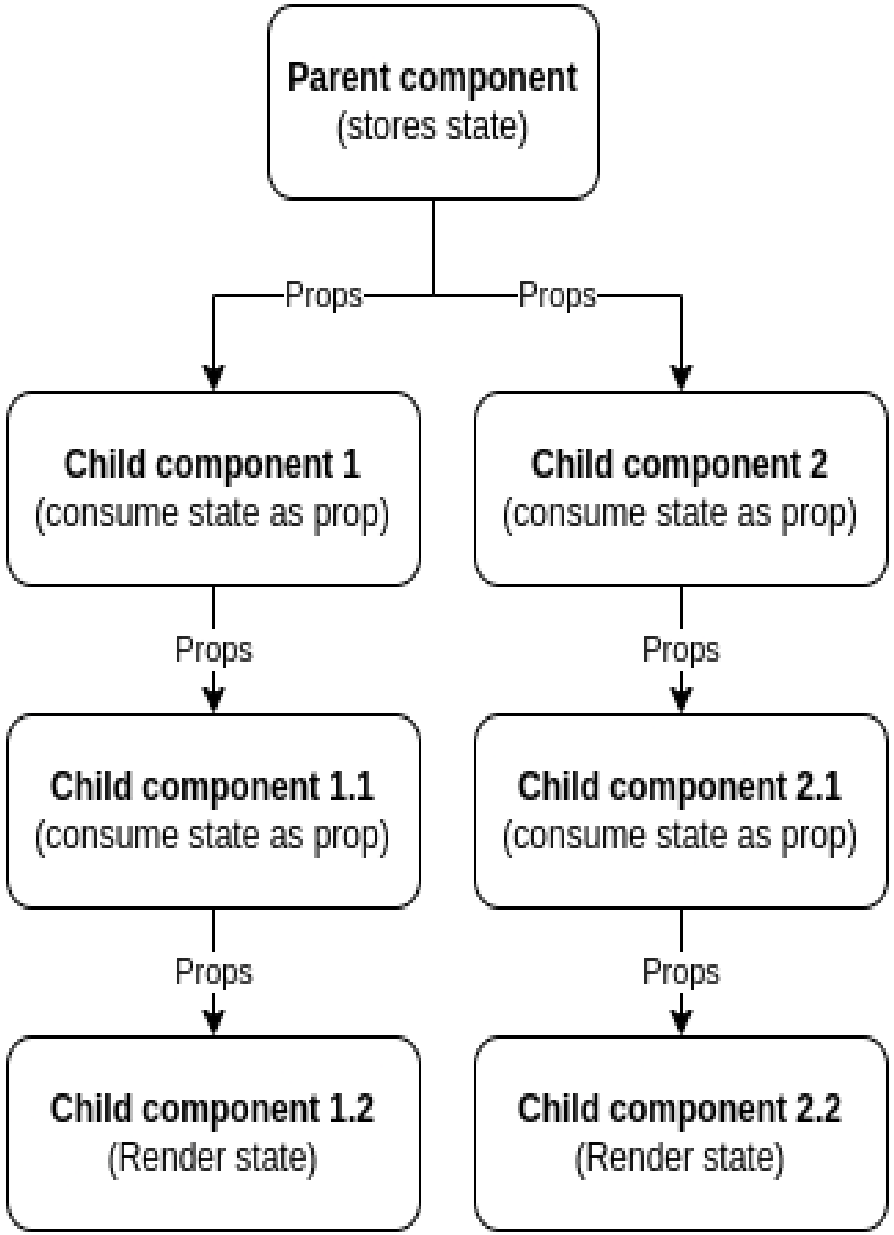
- Scenario where theres complex computed state that depends on multiple values, **useMemo** helps to manage the dependencies efficiently. It allows you to calculate the value once when any of the dependencies change.
- **useMemo** can also be used to memoize functions that don't depend on component state. This is particularly useful when passing callback functions as props to child components.

# useReducer()

- The useReducer Hook is used to store and update states, just like the useState Hook.
- It accepts a reducer function as its first parameter and the initial state as the second.
- useReducer returns an array that holds the current state value and a dispatch function to which user can pass an action and later invoke it.

# useContext()


Parent component
Update state
Context
(stores state)
Child component 1
Child component 2
Child component 1.1
Child component 2.1
Child component 1.2
(Render state)
Child component 2.2
(Render state)
Read state


Parent component
(stores state)
Props
Props
Child component 1
(consume state as prop)
Child component 2
(consume state as prop)
Props
Props
Child component 1.1
(consume state as prop)
Child component 2.1
(consume state as prop)
Props
Props
Child component 1.2
(Render state)
Child component 2.2
(Render state)

# contd..

- React context is a built-in feature that allows you to share data, such as a UI theme, authenticated user, or locale preferences, across your component tree without having to pass props manually at every level.
- **useContext** is a React hook for subscribing to React Context, allowing functional components to access shared data (state, functions) from a Provider component without prop drilling, making data flow cleaner by reading values directly from the nearest Provider up the component tree.
- It takes a context object (created with createContext) as an argument and returns the current context value, simplifying global state management for things like themes, user auth, or preferences.
- **Create Context**: Use React.createContext() to create a Context object, defining its structure and default value (e.g., const ThemeContext = createContext('light');).
- **Provide Context**: Wrap the part of your component tree that needs the data with <Context.Provider value={...}>, passing the data through the value prop.

# contd..

- **ReactNode** is a type representing anything that React can render on the screen. It is the most inclusive and flexible type for content that may be displayed in your application, commonly used for defining the type of a component's children prop.

## What a ReactNode includes

A ReactNode is a union type that can be one of the following:

- **React elements:** An object created via JSX (e.g., Hello ).
- **Primitive types:**
  - string (text).
  - number.
  - boolean (though true and false are ignored and not rendered).
  - null or undefined (which are also ignored and do not render anything).
- **Arrays/Iterables:** An array containing any combination of other ReactNodes (for rendering lists of items).
- **Fragments and Portals:** Other specific React constructs like React.Fragment or React.Portal.

# useCallback()

- **useCallback hook** returns a memoized function to reduce unnecessary callbacks. This useCallback hook is used when you have a component in which the child is rerendering again and again without need.

# contd..

- The components created of the two types are **equivalent**, and also the basic difference between a functional component and class component.
- **Functional component** is used only when the component does not require interacting or working with any other component i.e. these components do not require data from other components
- Multiple functional components can be composed under a single functional component.
- Class components can also be used for this purpose but it is not recommended if there is no need will make the application in-efficient.
- A class component can be called in a functional component in React. This is achieved by using the React Hooks API, specifically the useState and useEffect hooks.